Week-12

Karthik Thiagarajan

1. Loss functions for Classification 2
1.1. $0-1$ Loss 2
1.2. Convex Surrogates 3
1.2.1. Squared Loss 4
1.2.2. Hinge Loss 5
1.2.3. Logistic Loss 6
1.2.4. Perceptron Loss 7
2. Neural Network 8
2.1. Network Architecture 8
2.2. Activation functions 10
2.2.1. ReLU: Rectified Linear Unit 10
2.2.2. Sigmoid 11
2.3. Computation at a neuron 11
2.4. Forward Pass 13
2.4.1. Regression 14
2.4.2. Binary Classification 14
2.5. Backward Pass 15
2.6. Flow of information 15

1. Loss functions for Classification

1.1. $0-1$ Loss

Given a classifier $h : \mathbb{R}^d \to \{-1, 1\}$, the loss function used for determining the goodness of the classifier is:

$$\frac{1}{n}\sum_{i=1}^{n} \mathbb{I}[h(\mathbf{x}_i) \neq y_i]$$

This is the average number of points that are misclassified. The goal is to find a classifier that minimizes the $0-1$ loss:

$$\min_{h \in \mathcal{H}} \quad \frac{1}{n}\sum_{i=1}^{n} \mathbb{I}[h(\mathbf{x}_i) \neq y_i]$$

$\mathcal{H}$ is the class of functions that we will search in. Instead of looking at all possible functions, we can restrict the class of functions to linear functions:

$$\mathcal{H}_{\text{linear}} = \left\{ \text{sign}\left(\mathbf{w}^T\mathbf{x}\right) : \ \mathbf{w} \in \mathbb{R}^d \right\}$$

More elaborately, any linear classifier $h \in \mathcal{H}_{\text{linear}}$ is given as:

$$h(\mathbf{x}) = \text{sign}\left(\mathbf{w}^T\mathbf{x}\right) = \begin{cases} 1, & \mathbf{w}^T\mathbf{x} \geqslant 0 \\ -1, & \mathbf{w}^T\mathbf{x} < 0 \end{cases}$$

The optimization problem now becomes:

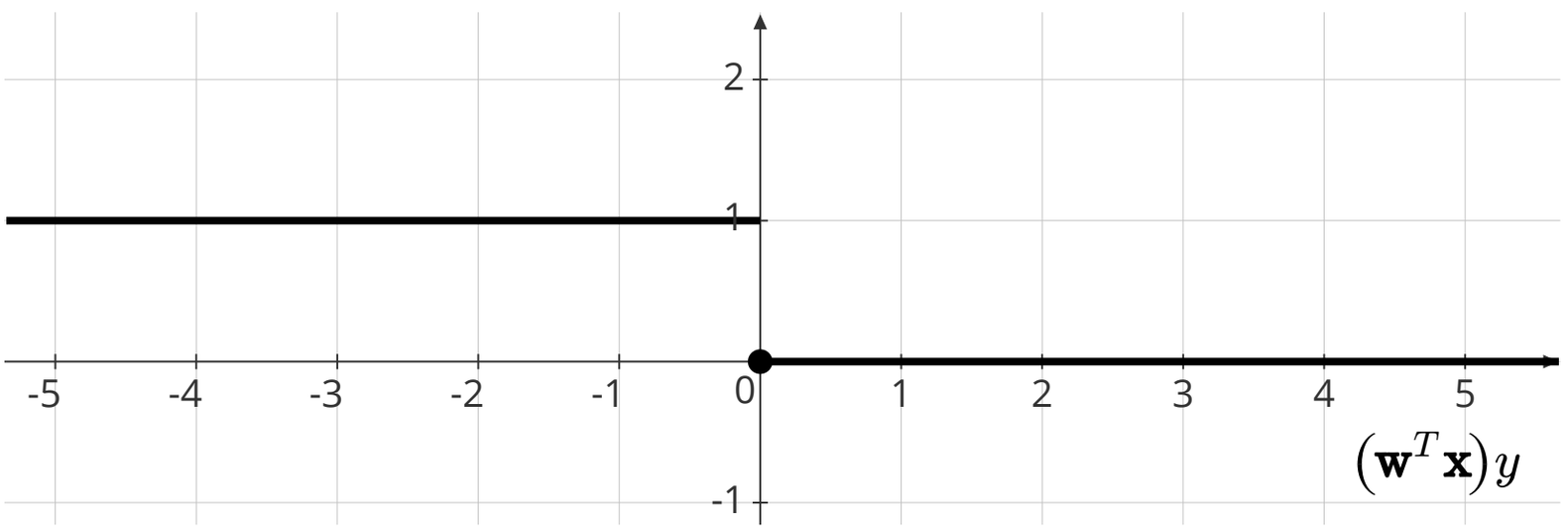
$$\min_{h \in \mathcal{H}_{\text{linear}}} \frac{1}{n} \sum_{i=1}^{n} \mathbb{I}[h(\mathbf{x}_i) \neq y_i]$$

Even though the set $\mathcal{H}_{\text{linear}}$ is simple enough, solving the above optimization problem is still (NP) hard. One way to mitigate the problems posed by the $0-1$ loss is to look for alternative loss functions that approximate the behavior of the $0-1$ loss but are easier to optimize. Convex functions fit this description.

For a single training data-point $(\mathbf{x}, y)$, the $0-1$ loss can be given as:

$$\mathbb{I}\Big[\big(\mathbf{w}^T\mathbf{x}\big)y < 0\Big]$$

Visually:



1.2. Convex Surrogates

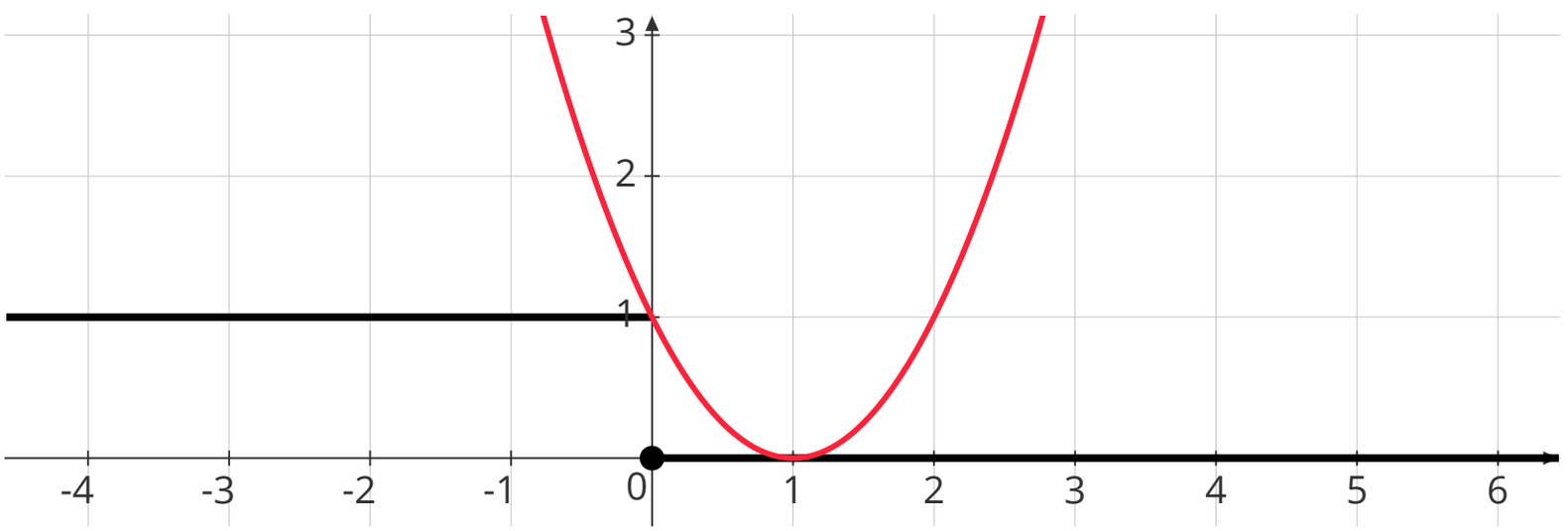
We will look at various convex approximations to the $0-1$ loss. Each such loss gives rise to a classification algorithm. For the rest of this document, we will only focus on linear classifiers. Since the quantity $\left(\mathbf{w}^T\mathbf{x}\right)y$ plays a major role, we will use the shorthand $u = \left(\mathbf{w}^T\mathbf{x}\right)y$. Note that $y \in \{-1, 1\}$.

1.2.1. Squared Loss

$$
\begin{aligned}
L(\mathbf{w}, \mathbf{x}, y) &= \left(\mathbf{w}^T\mathbf{x} - y\right)^2 \\
&= \left(\mathbf{w}^T\mathbf{x}\right)^2 + y^2 - 2\left(\mathbf{w}^T\mathbf{x}\right)y \\
&= \left(\mathbf{w}^T\mathbf{x}\right)y^2 + 1 - 2\left(\mathbf{w}^T\mathbf{x}\right)y \\
&= \left[\left(\mathbf{w}^T\mathbf{x}\right)y - 1\right]^2
\end{aligned}
$$

We can therefore express this as:

$$(u-1)^2$$



This loss is associated with a least squares classifier (inspired by regression). The loss is convex, but a poor approximation for the $0-1$ loss. It heavily penalizes even points that are correctly classified.

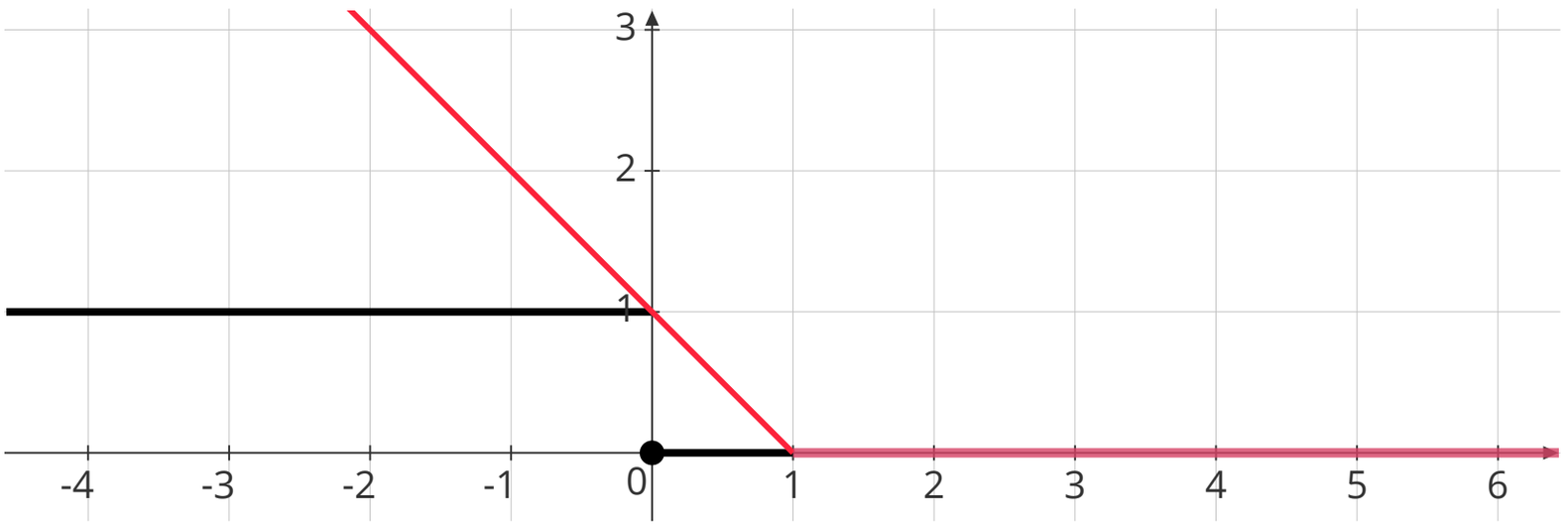
1.2.2. Hinge Loss

$$L(\mathbf{w}, \mathbf{x}, y) = \max\left(0, 1 - \left(\mathbf{w}^T\mathbf{x}\right)y\right)$$

This can be written as:

$$\max\left(0, 1 - u\right)$$

Plotting it:



The hinge loss is used in SVM. Notice that the loss penalizes points that are also correctly classified but violating the margin.

1.2.3. Logistic Loss

In logistic regression, the negative log-likelihood is treated as a loss function, also called the binary cross entropy.

$$-y\ln\Big[\sigma\big(\mathbf{w}^T\mathbf{x}\big)\Big] - (1-y)\ln\Big[1-\sigma\big(\mathbf{w}^T\mathbf{x}\big)\Big]$$

Here, the labels are in $\{0, 1\}$.

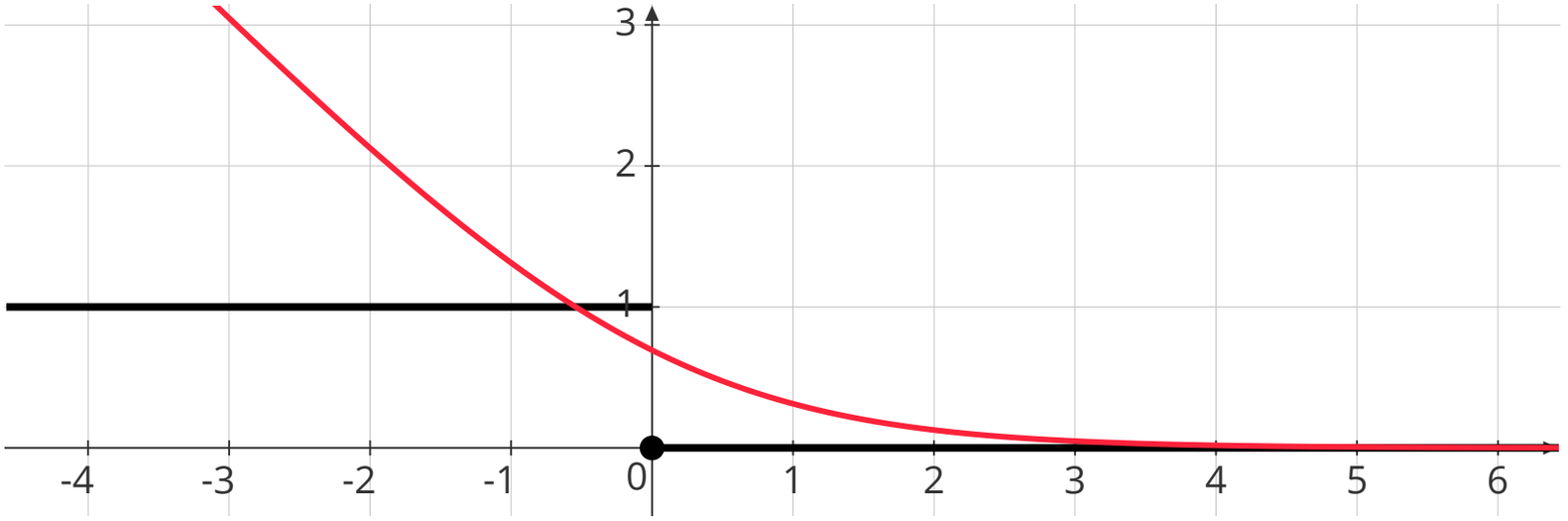
$$\begin{cases} -\ln\left(\dfrac{1}{1+e^{-\mathbf{w}^T\mathbf{x}}}\right), & y=1 \\ -\ln\left(\dfrac{1}{1+e^{\mathbf{w}^T\mathbf{x}}}\right), & y=0 \end{cases}$$

Since we are using $\{-1, 1\}$ as the label set, this loss becomes:

$$L(\mathbf{w}, \mathbf{x}, y) = \ln\Big(1+e^{-(\mathbf{w}^T\mathbf{x})y}\Big)$$

In terms of our notation:

$$\ln\big(1+e^{-u}\big)$$



This is the logistic loss and is associated with the logistic regression model.

1.2.4. Perceptron Loss

The perceptron update rule is:

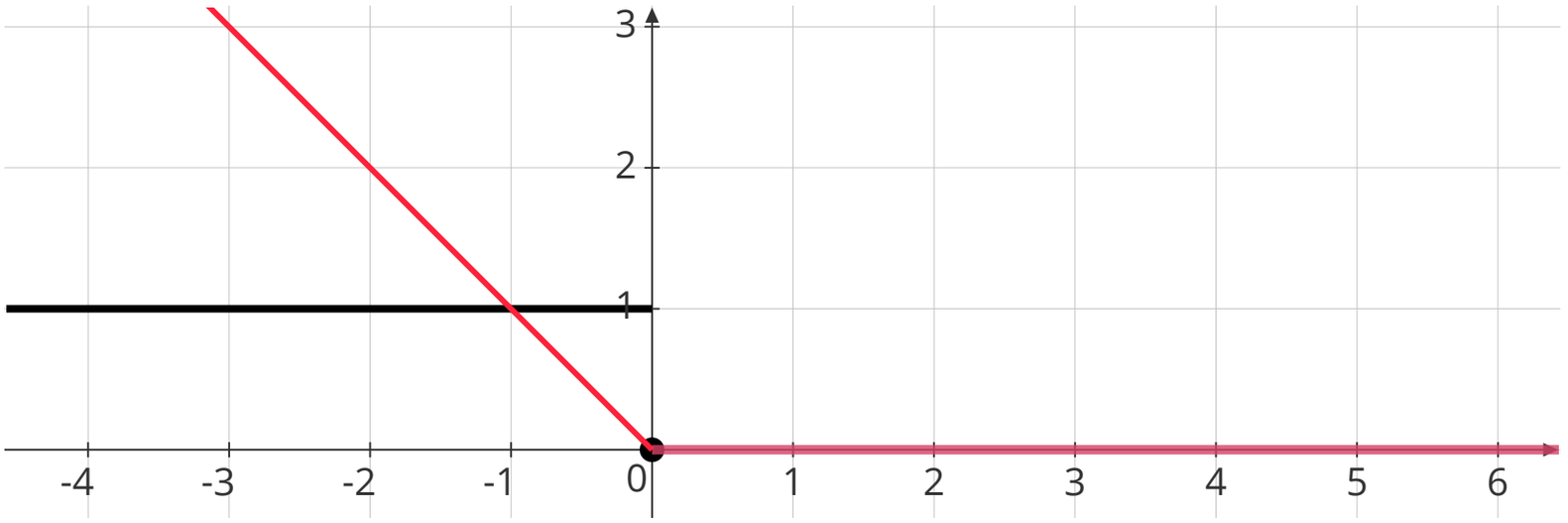
$$\mathbf{w} := \begin{cases} \mathbf{w} + \mathbf{x}y, & (\mathbf{w}^T\mathbf{x})y < 0 \\ \mathbf{w}, & (\mathbf{w}^T\mathbf{x})y > 0 \end{cases}$$

This can be viewed as SGD with a step size of one and a batch-size of one with the following loss:

$$L(\mathbf{w}, \mathbf{x}, y) = \max\left(0, -(\mathbf{w}^T\mathbf{x})y\right)$$

which becomes:

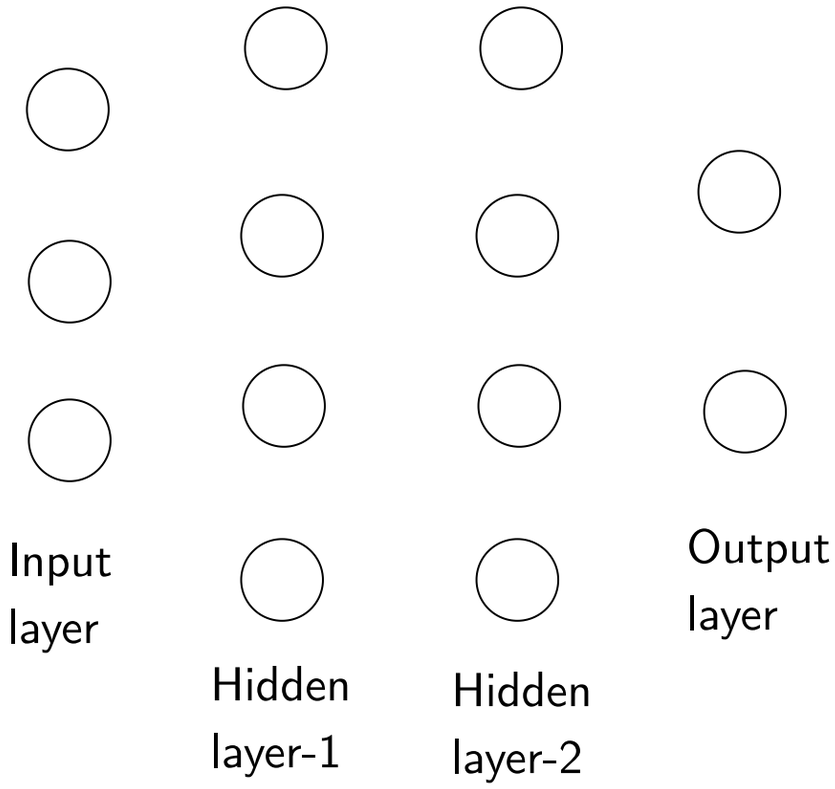
$$\max\left(0, -u\right)$$



2. Neural Network

A neural network is a powerful model that learns highly non-linear relationships between inputs and outputs. They are capable of modeling almost any dataset and are particularly suitable for unstructured datasets such as images and text. The last decade of ML research has seen a resurgence of these models. Neural networks are now the state of the art in almost every conceivable sub-field of AI.

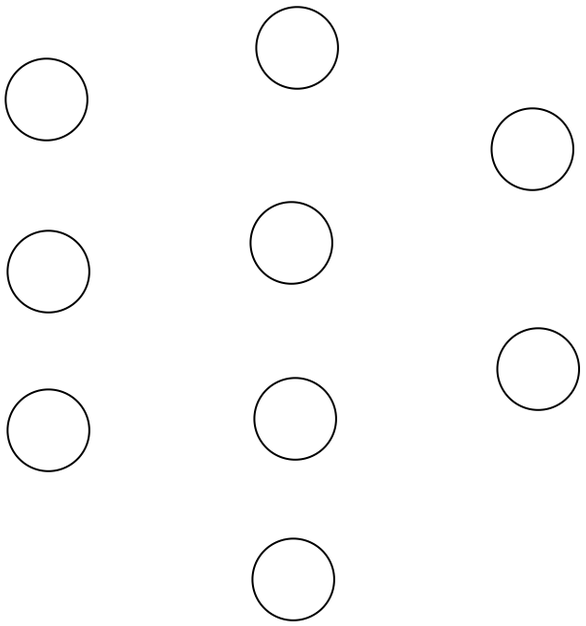
2.1. Network Architecture



There are several layers in a network:

- Input layer
- Hidden layers
- Output layer

The computational components in each layer are called artificial neurons or just neurons. Neurons in successive layers are connected by edges. Each edge has a weight associated with it. All neurons other than the input layer neurons have a bias associated with them. The total learnable parameters in a network is the sum of the number weights and biases across layers. For example, consider the network given below:



#weights$=3 \times 4 + 4 \times 2 = 20$

#biases$=6$

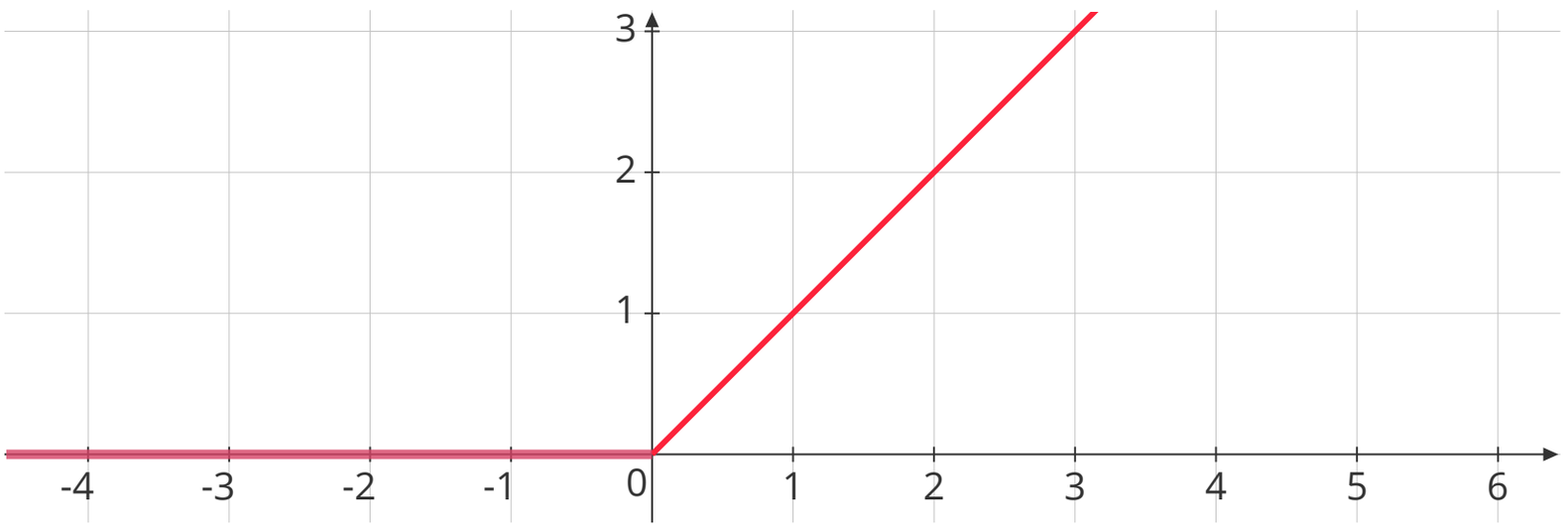
#parameters$=26$

2.2. Activation functions

Some non-linear computation happens at each neuron in a network. This is achieved with the help of what is called an activation function.

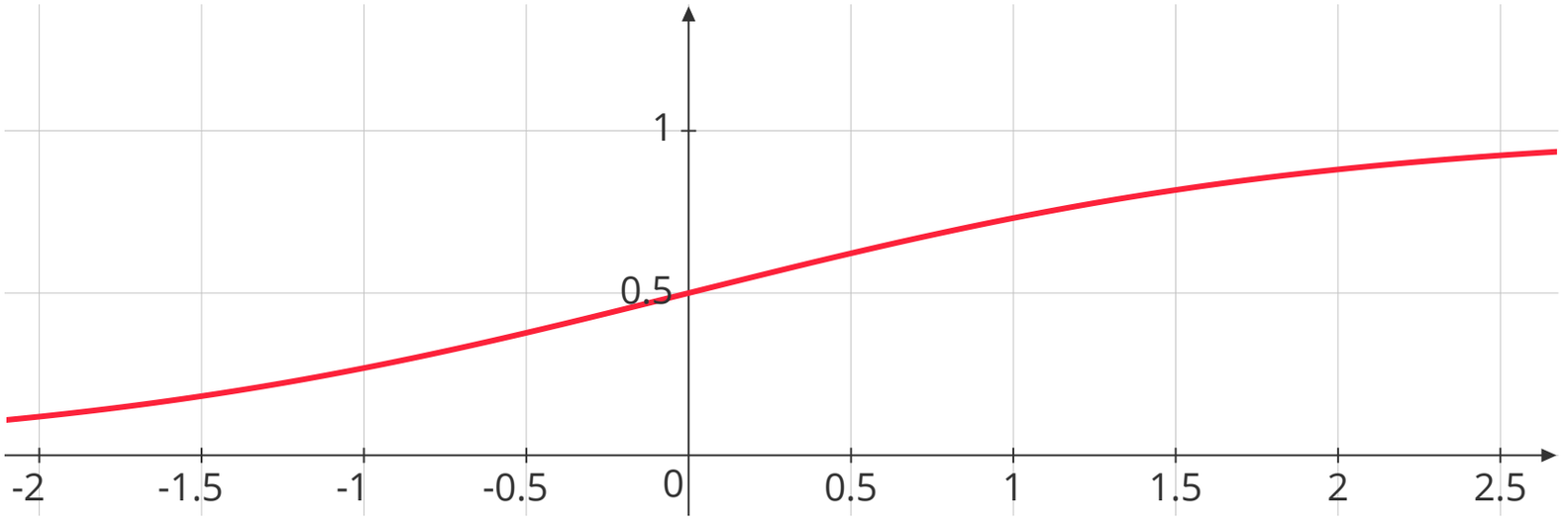
2.2.1. ReLU: Rectified Linear Unit

$$g(x) = \max(0, x) = \begin{cases} x, & x \geqslant 0 \\ 0, & x < 0 \end{cases}$$



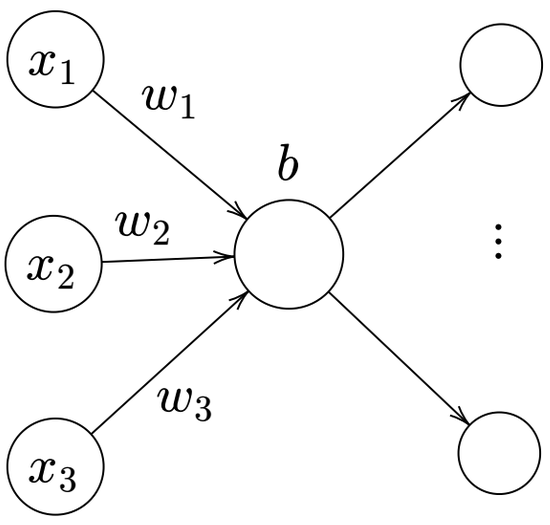
2.2.2. Sigmoid

$$g(x) = \frac{1}{1 + e^{-x}}$$



2.3. Computation at a neuron

Consider a single neuron at some layer:



The computation is composed of two steps:

- Linear combination of inputs from the previous layer
  - These are often called the pre-activation values
- Non-linear activation function over this linear combination
  - These are the activations (outputs at a neuron)

For example:

<u>Step-1: Linear combination</u>

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

Since z is the input to the activation function, z is called the pre-activation.

<u>Step-2: Non-linear activation</u>

$$g(z) = g(w_1x_1 + w_2x_2 + w_3x_3 + b)$$

$g(z)$ is called the activation. This is the output of the neuron and will be fed as input to neurons in the next layer.

If g is ReLU, the entire computation can be summarized as:

$$\text{ReLU}(w_1x_1 + w_2x_2 + w_3x_3 + b)$$

If g is sigmoid, the entire computation can be summarized as:

$$\sigma(w_1x_1 + w_2x_2 + w_3x_3 + b)$$

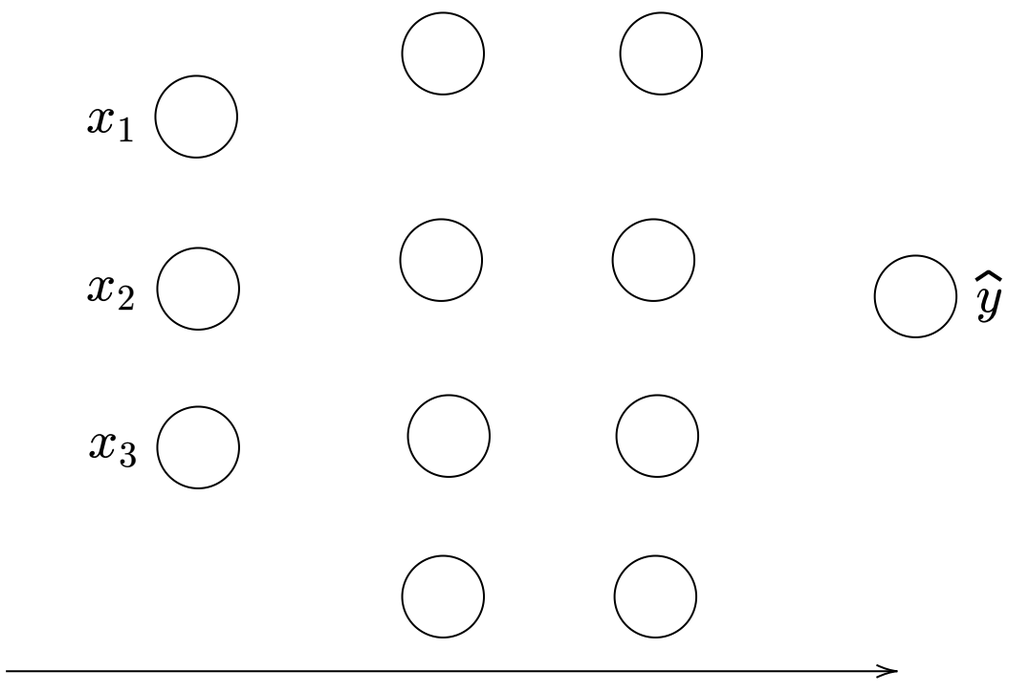
We often say that a neuron fires or is activated if it gives some non-zero output. For instance, in a ReLU neuron would fire if the pre-activations are positive. The

bias expresses the propensity of a neuron to fire or activate, independent of the inputs received. That is, if all inputs to a ReLU neuron are zero, the bias gives an idea of what would happen. If the bias is positive, it has a propensity to fire, if the bias is negative, it has a propensity to remain dormant unless the inputs are powerful enough to activate it.

2.4. Forward Pass

The forward pass in a network takes the input and produces an output. The network can be seen as a function. To keep things simple, consider a regression problem. Then, the network can be viewed as a non-linear function:

$$h : \mathbb{R}^d \to \mathbb{R}$$



The computations are propagated forward, from left to right, hence the term forward pass. Depending on the problem, the following things will change:

- number of neurons in the output layer

- choice of activation function at the output layer
- loss function

2.4.1. Regression

For a regression problem, the output layer will have one neuron and the activation function at the output neuron will be linear (identity) and the loss function will be the squared loss. If h is the neural network, we have:

$$\widehat{y} = h(\mathbf{x})$$

$$L(w) = (\widehat{y} - h(\mathbf{x}))^2$$

2.4.2. Binary Classification

There is some variability here, but for this course, we will assume that the output layer has one neuron. The activation function at the output neuron will be sigmoid. In this case, the output value will be interpreted as a probability:

$$h(\mathbf{x}) = \widehat{y} = P(y = 1 \mid \mathbf{x})$$

The loss function will be binary cross entropy loss.

$$-y \ln \widehat{y} - (1 - y) \ln(1 - \widehat{y})$$

2.5. Backward Pass

Gradient descent can be used to minimize the loss and update the weight vector. Combine all parameters of the network into one vector, say $\boldsymbol{\theta}$.

$$\boldsymbol{\theta}^{(t+1)} := \boldsymbol{\theta}^{(t)} - \eta \nabla L_{\boldsymbol{\theta}^{(t)}}$$

Computing the gradient in a neural network is done using an algorithm called back-propagation (chain-rule applied from the last layer to the input layer).

2.6. Flow of information

The flow of information in a neural network:

- Forward pass
  - Left to right
  - Input to output
- Loss function
  - True labels and output (predicted labels)
- Backward pass
  - Compute the gradient of the loss with respect to the weights
  - Update the weights/biases using gradient descent

Repeat this process for several iterations.